

## Code-Layout, Readability and Reusability

|                      |                                                |
|----------------------|------------------------------------------------|
| <b>Date</b>          | 30 September 2026                              |
| <b>Team ID</b>       | SWTID-2026-8026                                |
| <b>Project Name</b>  | Import Data using Transform maps (Spreadsheet) |
| <b>Maximum Marks</b> | 5 Marks                                        |

### Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

### Code Layout Checklist:

| S.No | Code Quality Parameter    | Description                                   | Followed (Yes / No / Partial) | Remarks                                                                                         |
|------|---------------------------|-----------------------------------------------|-------------------------------|-------------------------------------------------------------------------------------------------|
| 1    | Consistent Indentation    | Uniform spacing/tabs used throughout the code | Yes                           | Transform map scripts use uniform 4-space indentation.                                          |
| 2    | Proper File Structure     | Files and folders are logically organized     | Yes                           | Data source, import set table, transform map and field maps are organized under one update set. |
| 3    | Meaningful Variable Names | Variables reflect their purpose clearly       | Yes                           | Names like sourceRow, targetRecord and isDuplicate show their purpose.                          |
| 4    | Function / Method Names   | Functions are descriptively named             | Yes                           | Functions such as validateEmail() and formatDate() describe their task.                         |
| 5    | Code Comments             | Inline and block comments explain logic       | Yes                           | onBefore/onAfter scripts have comments explaining the logic.                                    |
| 6    | Modular Design            | Code is split into reusable functions/modules | Partial                       | Validation is in a Script Include; some logic is still inline in field map scripts.             |
| 7    | No Redundant Code         | Duplicate or unused code is removed           | Yes                           | Unused field maps removed; duplicates handled using Coalesce.                                   |
| 8    | Error Handling            | Exceptions and errors are handled gracefully  | Yes                           | Invalid rows are skipped (ignore = true) and logged with gs.error().                            |

**Reusable Components / Modules:**

| S.No | Component / Module Name          | Language / Technology    | Where Reused                                  | Reusability Level (High / Medium / Low) |
|------|----------------------------------|--------------------------|-----------------------------------------------|-----------------------------------------|
| 1    | Excel Data Source (.xlsx)        | ServiceNow Data Source   | Every spreadsheet upload / import             | High                                    |
| 2    | Import Set Table                 | ServiceNow Table         | Staging table for all imports                 | High                                    |
| 3    | Transform Map                    | ServiceNow Transform Map | Maps import set data to target table          | Medium                                  |
| 4    | Field Map Scripts (data cleanup) | JavaScript               | Date, email and text formatting in field maps | Medium                                  |
| 5    | Validation Script Include        | JavaScript               | onBefore script of all transform maps         | High                                    |

**Overall Code Quality Assessment:**

| Aspect                   | Rating (1-5) | Comments                                                                 |
|--------------------------|--------------|--------------------------------------------------------------------------|
| Code Layout & Structure  | 4            | Well organized; consistent indentation and logical grouping.             |
| Readability              | 4            | Clear variable and function names; easy to follow.                       |
| Reusability              | 4            | Script Include and data source can be reused; some inline logic remains. |
| Documentation / Comments | 4            | Scripts are commented; can add more detail on field mappings.            |
| Overall Score            | 4            | Good quality, maintainable and reusable.                                 |